

Dokumentacja Techniczna i Projektowa: Embedded Bird Classifier

Autorzy projektu: Emil Siatka, Mateusz Szwagierczak

Data wydania: Czerwiec 2026 r.

Wersja systemu: 1.0.0

Platforma bazowa: Raspberry Pi 4 Model B (2GB RAM)

System operacyjny: Raspberry Pi OS (64-bit, Debian Bookworm based)

1. Wstęp i Cel Projektu

Projekt **Embedded Bird Classifier** to autonomiczne urządzenie wbudowane przeznaczone do stałego nagrywania dźwięków otoczenia i automatycznego rozpoznawania gatunków ptaków. System został zaprojektowany do pracy w trudnych warunkach terenowych, bez dostępu do sieci internetowej oraz stałego zasilania z gniazdka.

Główne założenia funkcjonalne:

- Cykliczne nagrywanie próbek dźwiękowych z otoczenia za pomocą podłączonego mikrofonu.
- Przetwarzanie i klasyfikacja nagrań w czasie rzeczywistym bezpośrednio na Raspberry Pi za pomocą modelu BirdNet.
- Uruchomienie własnego Hotspot's, co pozwala na pobieranie danych i podgląd wyników na smartfonie lub komputerze bez użycia internetu.
- Zapisywanie pociętych próbek audio oraz logów w plikach JSON, z możliwością pobrania ich jako skompresowane archiwum ZIP i zdalnego czyszczenia pamięci flash.

2. Specyfikacja i Architektura Sprzętowa (Hardware)

Urządzenie ma budowę modułową opartą na łatwo dostępnych komponentach, co ułatwia szybkie zreplikowanie konstrukcji.

Komponent	Model / Producent	Rola w systemie	Specyfikacja techniczna / Uwagi
-----------	----------------------	--------------------	---------------------------------------

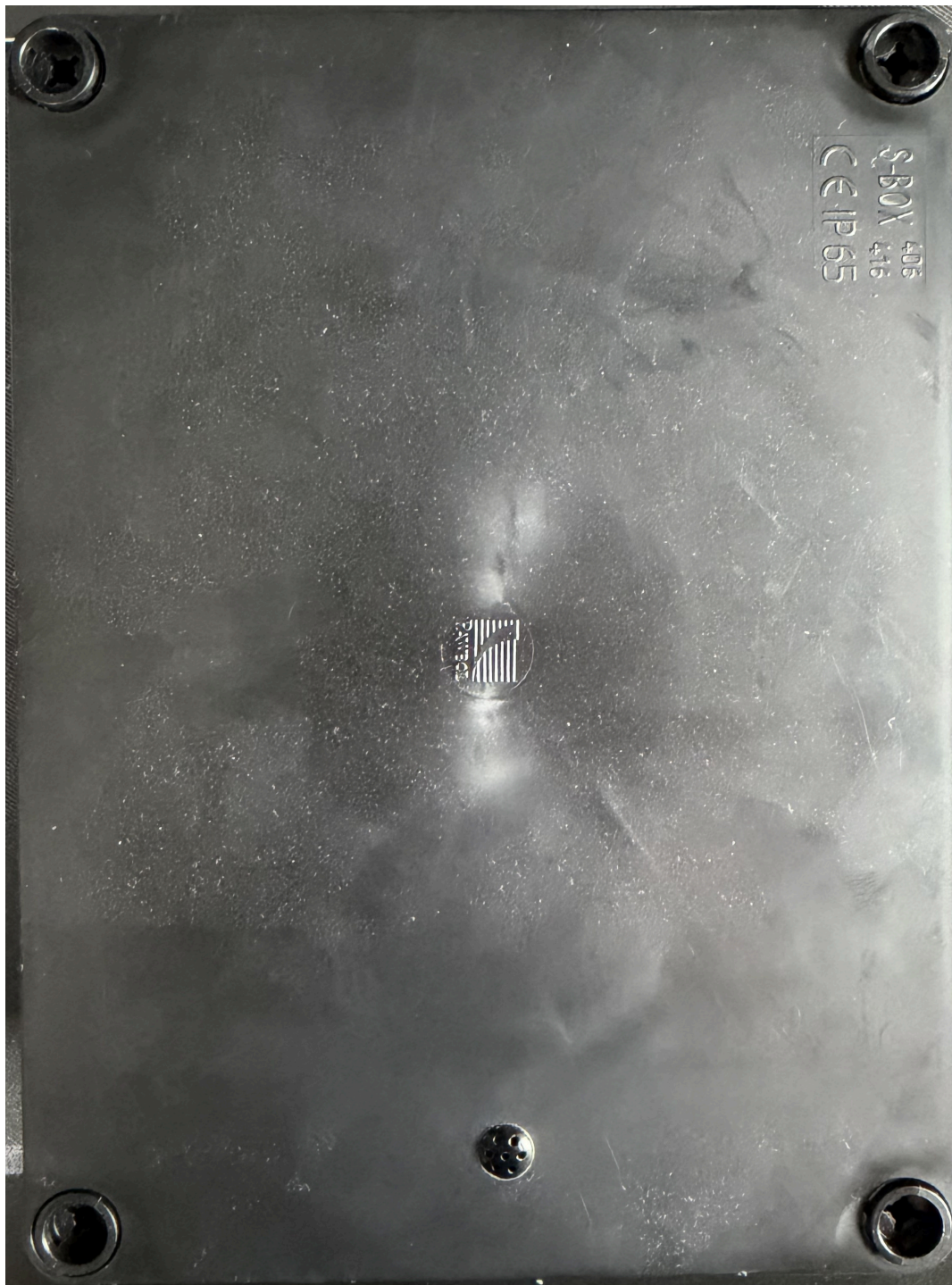
Jednostka centralna	Raspberry Pi 4 Model B	Główny minikomputer, uruchamianie procesów, analiza audio, serwer API/WWW.	Broadcom BCM2711 (Quad-core Cortex-A72 @1.5GHz), 2GB RAM LPDDR4, wbudowane Wi-Fi 2.4/5.0 GHz.
Układ zasilania	Powerbank Xiaomi	Autonomiczne źródło energii, funkcja bufora UPS.	Wspiera jednoczesne ładowanie powerbanka i zasilanie minikomputera z zewnętrznego źródła (<i>pass-through charging</i>).
Mikrofon	Mikrofon krawatowy Esperanza	Przechwytywanie odgłosów ptaków z otoczenia.	Charakterystyka wielokierunkowa, pasmo przenoszenia dopasowane do rejestracji dźwięków natury.
Karta dźwiękowa	Karta dźwiękowa USB LogiLink	Zamiana sygnału analogowego z mikrofonu na cyfrowy.	Działa w standardzie Plug&Play w systemie Linux, dedykowane wejście mikrofonowe TRS 3.5 mm.
Obudowa zewnętrzna	Puszka elektroinstalacyjna S-BOX (Pawbol)	Ochrona komponentów przed deszczem i wilgocią.	Klasa szczelności IP65, wykonana z wytrzymałego polipropylenu.

Montaż i uszczelnienie:

Komponenty wewnątrz puszki S-BOX zostały unieruchomione przy użyciu kabli zasilających oraz trytytek, co zabezpiecza je przed przemieszczaniem się w trakcie transportu. Kabel mikrofonowy wyprowadzono na zewnątrz

przez fabryczny przepust kablowy. Miejsce wyprowadzenia oraz łączenia obudowy uszczelniono klejem na gorąco, chroniąc wnętrze przed wilgocią. Na zewnątrz puszki mikrofon osłonięto perforowaną maskownicą ochronną.

Zdjęcia komponentów sprzętowych:



Rysunek 1: Zamknięta obudowa S-BOX z wyprowadzonym na dole mikrofonem.



Rysunek 2: Rozmieszczenie Raspberry Pi 4, powerbanka i karty dźwiękowej wewnątrz obudowy.

3. Architektura Oprogramowania i Stos Technologiczny

System składa się z niezależnych usług systemowych oraz modułów aplikacyjnych, które współpracują ze sobą w tle.

Stos Technologiczny (Tech Stack):

- **System Operacyjny:** Raspberry Pi OS (64-bit). Funkcje oszczędzania energii zostały wyłączone (`systemctl mask dla acpi/sleep`), aby urządzenie nie przechodziło w stan uśpienia w terenie.
- **Backend:** Python 3.11+ działający w środowisku wirtualnym (venv). Główne biblioteki: FastAPI (obsługa komunikacji z frontendem), uvicorn (serwer WWW), birdnetlib (obsługa modelu rozpoznawania BirdNet TensorFlow Lite), watchdog (automatyczne wykrywanie nowych plików audio), sounddevice i scipy (nagrywanie dźwięku i zapis do WAV), pydub (wycinanie fragmentów audio).
- **Frontend:** Aplikacja typu SPA zbudowana w React z użyciem narzędzia Vite. Wygląd ostyleowano za pomocą TailwindCSS, ikony pochodzą z pakietu Lucide React, a wykresy generuje biblioteka Recharts.

Struktura Katalogów Projektu:


```

/home/user/bird_classifier/
├── autorun_logs.md          # Logi diagnostyczne autostart
├── bird_images/            # Lokalna baza zdjęć ptaków do
├── .env                    # Konfiguracja i zmienne środowiska
├── frontend/              # Kod źródłowy aplikacji React
│   ├── index.html
│   ├── package.json
│   ├── tailwind.config.js
│   └── src/
│       ├── App.jsx        # Główny kod panelu użytkownika
│       ├── main.jsx
│       └── index.css
├── frontend.log            # Logi działania serwera Vite
├── Raspberry_config-PL.md # Instrukcja instalacji systemu
├── requirements.txt        # Lista bibliotek Pythona do zainstalowania
├── run.sh                  # Skrypt startowy uruchamiający aplikację
├── setup.sh                # Skrypt do automatycznej konfiguracji
├── src/                    # Skrypty backendowe (Core)
│   ├── __init__.py
│   ├── analyzer.py        # Rozpoznawanie ptaków i wykrywanie
│   ├── log_reader.py      # Wycinanie fragmentów audio (
│   ├── recorder.py        # Ciągłe nagrywanie dźwięku z
│   └── server.py          # Serwer FastAPI i przesyłanie danych
├── running/                # Folder roboczy (Runtime)
│   ├── export.zip         # Tymczasowa paczka ZIP do pobrania
│   ├── analyzing_results/ # Pliki z wynikami rozpoznawania
│   ├── new_audio_samples/ # Bufor na surowe, 9-sekundowe próbki
│   └── saved_audio_samples/ # Foldery sesji z pociętymi próbkami

```

Poniższy zrzut ekranu przedstawia poprawnie skonstruowaną strukturę plików oraz nadane uprawnienia do uruchamiania skryptów .sh:

```

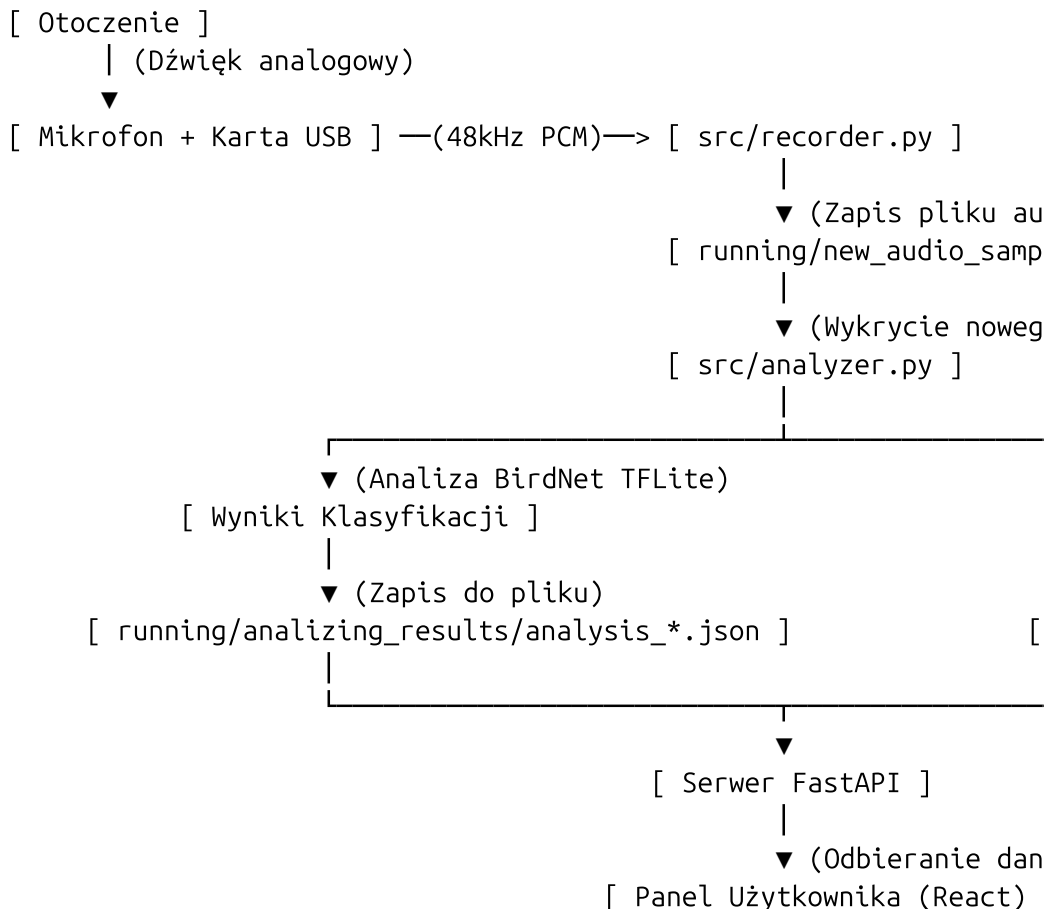
user@RaspberrySW:~$ ls -l bird_classifier/
total 88
-rwxrwxrwx 1 user user 4339 Apr 21 02:25 autorun_logs.md
drwxrwxrwx 2 user user 12288 May 27 18:33 bird_images
drwxrwxrwx 4 user user 4096 Jun 10 15:59 frontend
-rwxrwxrwx 1 user user 172 Jun 10 15:59 frontend.log
-rwxrwxrwx 1 user user 1085 May 27 18:27 LICENSE
-rwxrwxrwx 1 user user 6334 May 27 18:27 Raspberry_config.md
-rwxrwxrwx 1 user user 6551 May 27 18:27 Raspberry_config-PL.md
-rwxrwxrwx 1 user user 26 May 27 18:27 README.md
-rwxrwxrwx 1 user user 1304 May 27 18:27 requirements.txt
drwxrwxrwx 5 user user 4096 May 28 13:47 running
-rwxrwxrwx 1 user user 1826 May 27 20:09 run.sh
-rwxrwxrwx 1 user user 1767 May 27 18:27 setup.sh
drwxrwxrwx 3 user user 4096 May 27 19:58 src
drwxrwxrwx 6 user user 4096 May 27 20:47 .env
user@RaspberrySW:~$

```

Rysunek 3: Widok struktury plików i uprawnień wykonawczych w terminalu.

4. Przepływ Danych w Systemie (Data Flow)

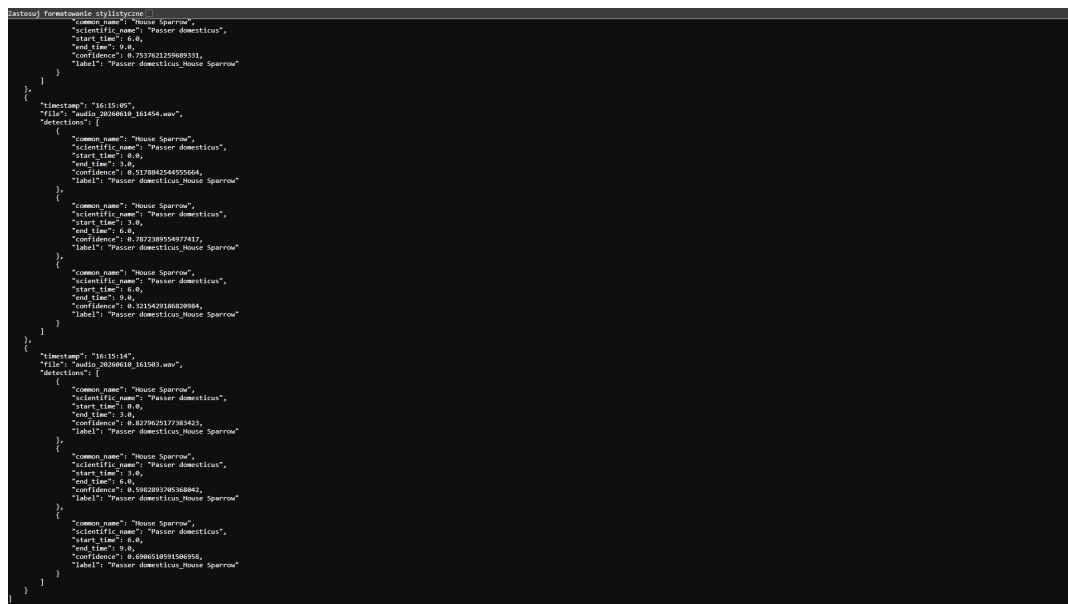
Urządzenie przetwarza dane w sposób ciągły, według poniższego schematu:



1. **Nagrywanie dźwięku:** Skrypt recorder.py uruchamia nagrywanie z częstotliwością 48 kHz w trybie mono. Co 9 sekund zapisuje zawartość do pliku audio_[TIMESTAMP].wav w folderze tymczasowym new_audio_samples.
2. **Wykrywanie plików:** Skrypt analyzer.py za pomocą modułu watchdog czuwa nad folderem new_audio_samples. Gdy pojawi się tam nowy plik .wav, skrypt odczeka sekundę (aby upewnić się, że plik został zapisany w całości), a następnie przekazuje go do analizy AI.
3. **Rozpoznawanie ptaków:** Biblioteka BirdNetLib uruchamia lokalny model TensorFlow Lite. Do analizy przekazywane są współrzędne geograficzne (domyślnie Warszawa: lat=52.2297, lon=21.0122), co pozwala odrzucić ptaki, które nie występują w naszym regionie i zwiększa dokładność rozpoznawania. Współrzędne można zmienić w pliku .env.

4. **Zapis i wycinanie próbek:** Jeśli model rozpozna ptaka z odpowiednią pewnością, dane trafiają do pliku `analysis_[SESJA].json`. Jednocześnie moduł `log_reader.py` (przy użyciu `pydub`) wycina z 9-sekundowego nagrania dokładnie ten moment, w którym ptak śpiewał, i zapisuje go jako osobny mały plik `.wav`.

Struktura pliku JSON zawiera pełne informacje o rozpoznanym gatunku oraz pewności predykcji:



```

{
  "timestamp": "16:15:07",
  "file": "audio_20200610_161454.wav",
  "detections": [
    {
      "common_name": "House Sparrow",
      "scientific_name": "Passer domesticus",
      "start_time": 0.0,
      "end_time": 3.0,
      "confidence": 0.727621250689331,
      "label": "Passer domesticus_House Sparrow"
    },
    {
      "common_name": "House Sparrow",
      "scientific_name": "Passer domesticus",
      "start_time": 0.0,
      "end_time": 3.0,
      "confidence": 0.517884154555666,
      "label": "Passer domesticus_House Sparrow"
    },
    {
      "common_name": "House Sparrow",
      "scientific_name": "Passer domesticus",
      "start_time": 3.0,
      "end_time": 6.0,
      "confidence": 0.7872389554977437,
      "label": "Passer domesticus_House Sparrow"
    },
    {
      "common_name": "House Sparrow",
      "scientific_name": "Passer domesticus",
      "start_time": 0.0,
      "end_time": 0.0,
      "confidence": 0.3215429186828984,
      "label": "Passer domesticus_House Sparrow"
    }
  ]
},
{
  "timestamp": "16:15:14",
  "file": "audio_20200610_161503.wav",
  "detections": [
    {
      "common_name": "House Sparrow",
      "scientific_name": "Passer domesticus",
      "start_time": 0.0,
      "end_time": 3.0,
      "confidence": 0.827962517783423,
      "label": "Passer domesticus_House Sparrow"
    },
    {
      "common_name": "House Sparrow",
      "scientific_name": "Passer domesticus",
      "start_time": 3.0,
      "end_time": 6.0,
      "confidence": 0.382289176168442,
      "label": "Passer domesticus_House Sparrow"
    },
    {
      "common_name": "House Sparrow",
      "scientific_name": "Passer domesticus",
      "start_time": 0.0,
      "end_time": 9.0,
      "confidence": 0.69605180106026,
      "label": "Passer domesticus_House Sparrow"
    }
  ]
}

```

Rysunek 4: Widok struktury pliku JSON wysyłanego do aplikacji frontendowej.

5. **Wyświetlanie wyników:** Serwer FastAPI udostępnia endpointy API oraz pozwala na pobieranie plików. Panel użytkownika w React odpytuje serwer co 5 sekund, automatycznie odświeżając wykresy i listę nagrań na ekranie.

5. Implementacja Backend (Python)

5.1. `src/recorder.py`

Ten skrypt odpowiada za obsługę karty dźwiękowej USB i nagrywanie audio.

```

# Mechanizm nagrywania dźwięku do bufora i zapisu na dysk
def record(device_id):
    samplerate = 48000
    channels = 1
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    file_path = os.path.join(FOLDER_AUDIO, f"audio_{timestamp}.wav")
    recording_buffer = []

    def callback(indata, frames, time_info, status):
        # Obliczanie głośności i wyświetlanie prostego paska VU w
        volume_norm = np.linalg.norm(indata) * 20 / np.sqrt(len(indata))
        recording_buffer.append(indata.copy())
        level = min(int(volume_norm * BAR_LENGTH), BAR_LENGTH)
        bar = "█" * level + "-" * (BAR_LENGTH - level)
        print(f"\rRecording (ID:{device_id}): [{bar}]", end="", flush=True)

    with sd.InputStream(samplerate=samplerate, channels=channels,
                       callback=callback, device=device_id):
        time.sleep(RECORDING_DURATION)

    full_recording = np.concatenate(recording_buffer, axis=0)
    write(file_path, samplerate, full_recording)

```

5.2. src/analizer.py

Główny skrypt odpowiedzialny za analizę audio i koordynację zapisu wyników.


```

class BirdWatchHandler(FileSystemEventHandler):
    def on_created(self, event):
        if event.is_directory or not event.src_path.lower().endswi
            return

        time.sleep(1) # Czekamy na zakończenie zapisu pliku na kar
        try:
            recording = Recording(analyzer, event.src_path, lat=52
            recording.analyze()

            if recording.detections:
                # Zapisywanie nowych wykryć na koniec pliku JSON
                if not os.path.exists(file_path):
                    with open(file_path, "w", encoding="utf-8") as

            record_data = {
                "timestamp": os.path.basename(event.src_path)[
                "file": os.path.basename(event.src_path),
                "detections": recording.detections
            }

            with open(file_path, "r+", encoding="utf-8") as f:
                data = json.load(f)
                data.append(record_data)
                f.seek(0)
                json.dump(data, f, indent=4, ensure_ascii=False
                f.truncate()

                # Wycięcie fragmentu audio z głosem ptaka
                segment_audio_parts(recording.detections, event.sr
        except Exception as e:
            print(f"Błąd analizy: {e}")

```

5.3. src/server.py

Serwer FastAPI udostępnia interfejs API dla aplikacji WWW, umożliwia pobranie spakowanych danych i czyszczenie karty pamięci.

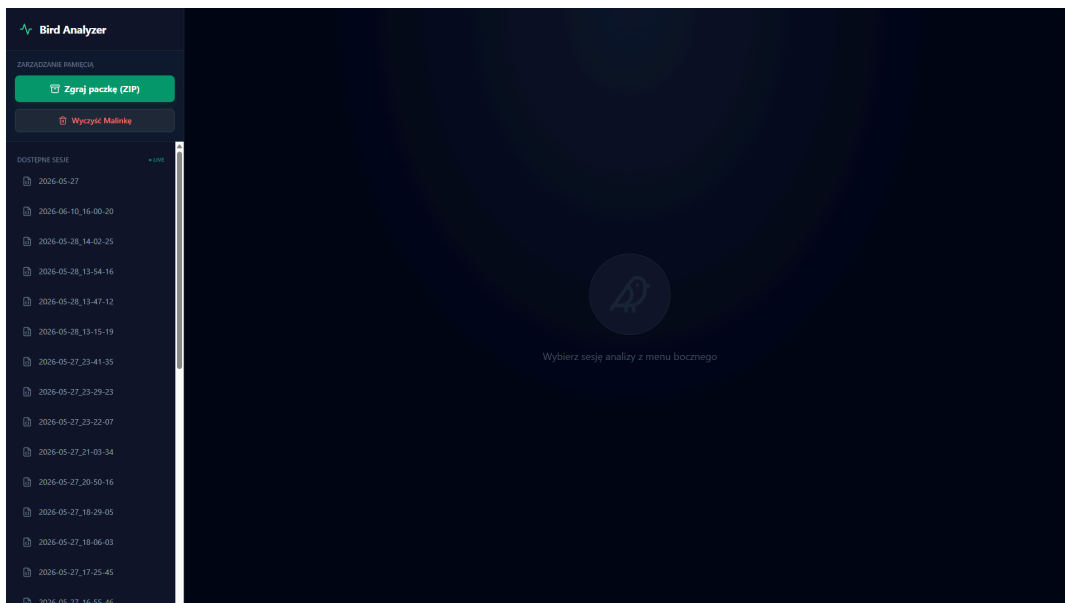
- GET /api/results: Zwraca listę zapisanych sesji (plików .json), od najnowszych do najstarszych.
 - GET /api/export: Pakuje w locie wszystkie wyniki i wycięte próbki audio do jednego archiwum ZIP, ułatwiając pobranie danych na raz.
 - DELETE /api/clear: Usuwa całą zawartość folderów z wynikami i nagraniami z karty pamięci, przygotowując system na nową sesję pomiarową.
-

6. Interfejs Użytkownika (Frontend - React)

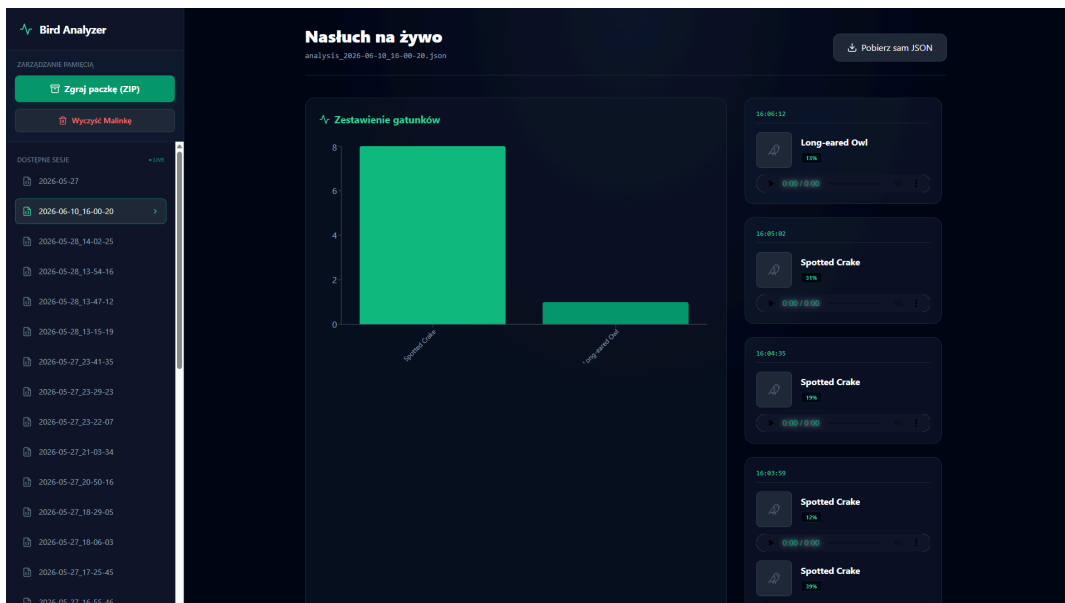
Główne mechanizmy w App.jsx:

1. **Obsługa odświeżania w tle:** Użyto referencji useRef, aby uruchomiony interwał (setInterval co 5 sekund) zawsze wiedział, którą sesję użytkownik aktualnie przegląda. Zapobiega to resetowaniu widoku i migotaniu ekranu podczas automatycznego pobierania nowych danych.
2. **Przygotowanie danych do wykresu:** Funkcja generateChartData zlicza wystąpienia poszczególnych gatunków ptaków w wybranej sesji i przekazuje te dane do wykresu słupkowego <BarChart> z biblioteki recharts.
3. **Brak problemów z pamięcią podręczną:** Adresy URL plików audio i raportów JSON są uzupełniane o zmienną ze znacznikiem czasu (?t=\${new Date().getTime()})). Dzięki temu przeglądarki na telefonach nie zapamiętują starych wersji plików i zawsze odtwarzają aktualne nagrania.

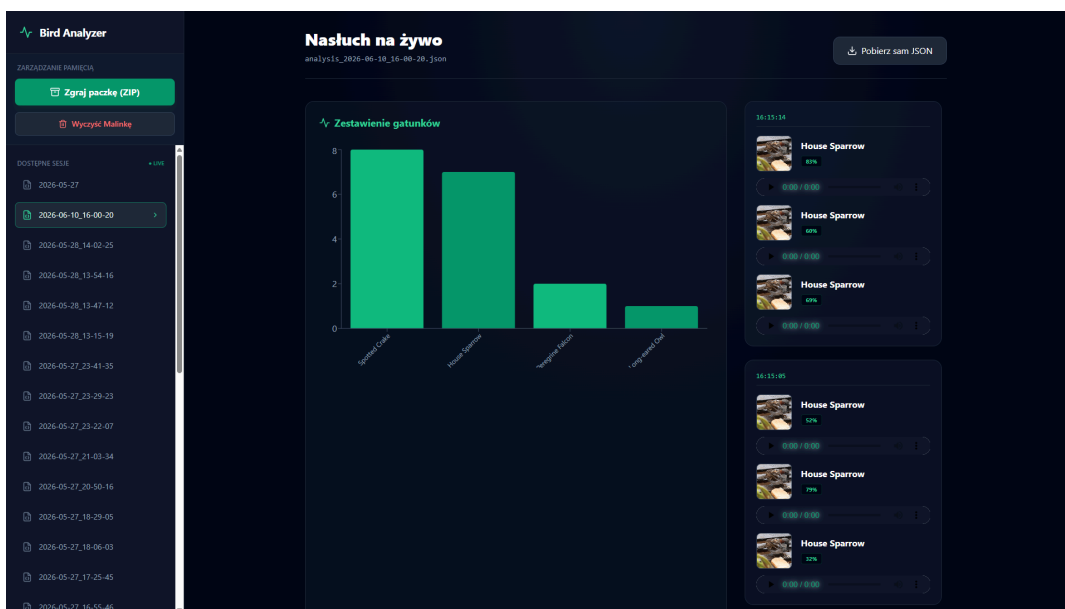
Wygląd panelu użytkownika:



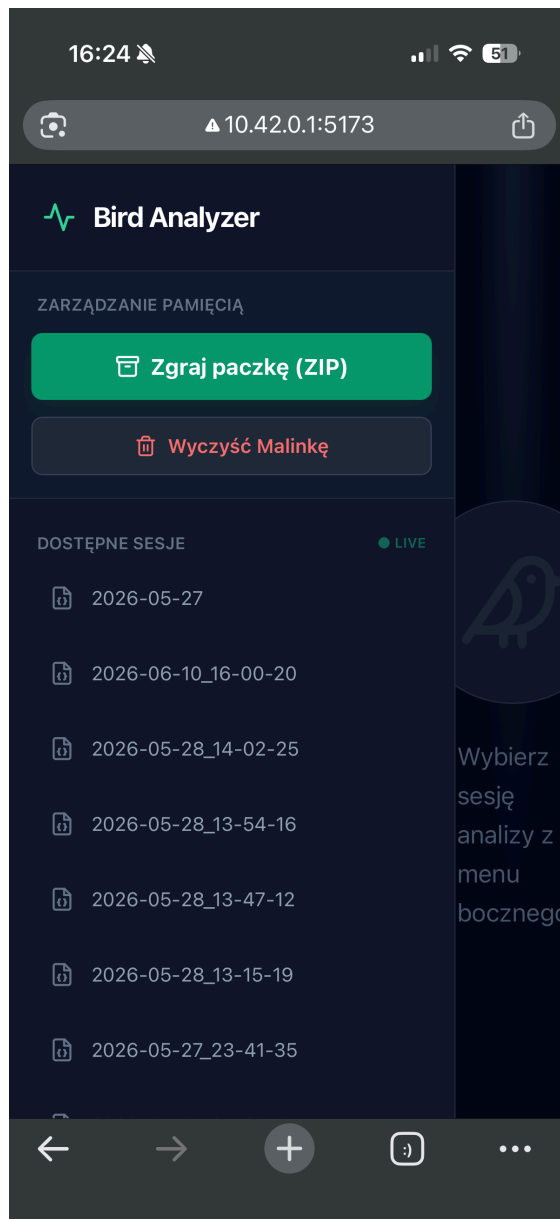
Rysunek 5: Widok panelu głównego przed wybraniem sesji.



Rysunek 6: Widok sesji z wykresem słupkowym i listą wykrytych ptaków.



Rysunek 7: Wyświetlanie zdjęcia rozpoznanego gatunku z lokalnej bazy danych.



Rysunek 8: Widok panelu zarządzania na ekranie smartfona.

7. Środowisko Systemowe, Autostart i Wdrożenie Terenowe

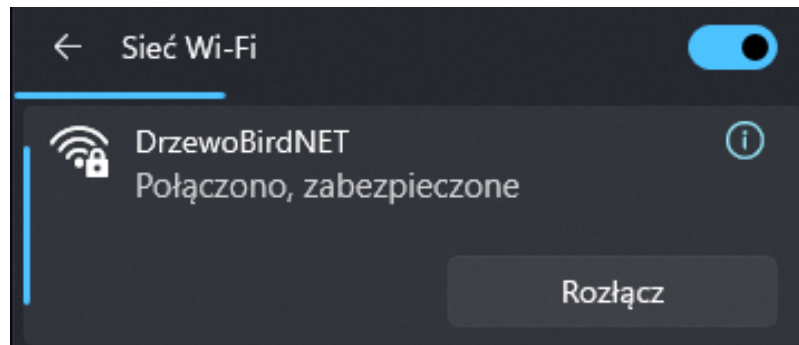
Urządzenie działa w pełni samodzielnie dzięki odpowiedniej konfiguracji usług systemowych i sieci na poziomie systemu operacyjnego Linux.

7.1. Konfiguracja sieci (Hotspot w terenie)

Za pomocą narzędzia NetworkManager (`nmcli`) karta Wi-Fi malinki (`wlan0`) została skonfigurowana jako punkt dostępowy:

```
sudo nmcli device wifi hotspot ifname wlan0 ssid DrzewoBirdNET pas  
sudo nmcli connection modify Hotspot connection.autoconnect yes
```

Malinka ma w tej sieci stały adres IP: 10.42.0.1.



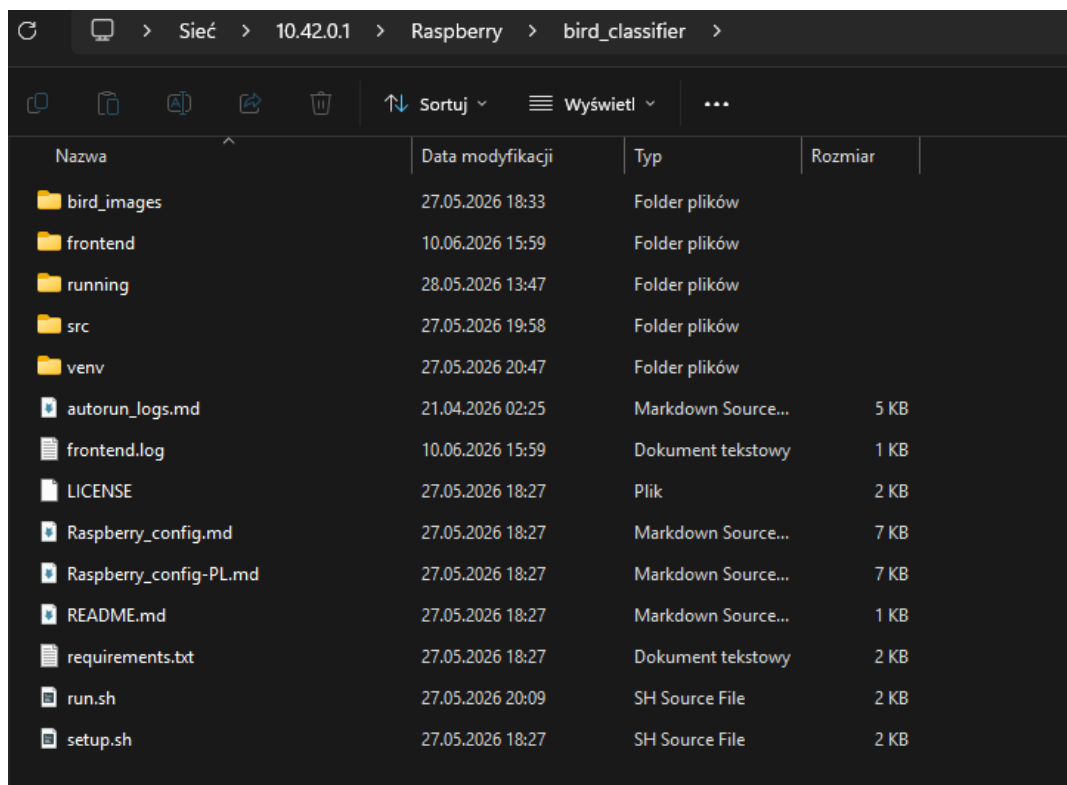
Rysunek 9: Połączenie z siecią Wi-Fi generowaną przez urządzenie.

7.2. Udostępnianie plików przez Sambę

Aby ułatwić bezpośredni dostęp do plików z komputera, skonfigurowano serwer Samba (smbd). W pliku `/etc/samba/smb.conf` dodano wpis:

```
[Raspberry]  
path = /home/user  
writeable = yes  
browseable = yes  
public = no  
create mask = 0644  
directory mask = 0755  
force user = user
```

Dzięki temu katalog domowy urządzenia można zamontować jako dysk sieciowy:



Rysunek 10: Dostęp do plików projektu z poziomu Eksploratora Windows.

7.3. Automatyczne uruchamianie procesów (Systemd)

Usługa `birdnet.service` dba o to, aby skrypt startowy `run.sh` włączał się samoczynnie od razu po uruchomieniu malinki (gdy tylko załadują się systemy audio i sieciowe), a także w razie błędu uruchamiał procesy ponownie.

Plik konfiguracyjny usługi (`/etc/systemd/system/birdnet.service`):

```
[Unit]
Description=Embedded Bird Classifier
After=network.target sound.target

[Service]
Type=simple
User=user
WorkingDirectory=/home/user/bird_classifier
ExecStart=/bin/bash /home/user/bird_classifier/run.sh
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Nagrania z poszczególnych dni są zapisywane w osobnych folderach, co ułatwia zachowanie porządku:


```
user@RaspberrySW:~ $ tree -d bird_classifier/running/
bird_classifier/running/
├── analyzing_results
├── new_audio_samples
├── saved_audio_samples
│   ├── 2026-05-27_19-58-36
│   ├── 2026-05-27_20-00-21
│   ├── 2026-05-27_20-06-28
│   ├── 2026-05-27_20-10-18
│   ├── 2026-05-27_20-12-54
│   ├── 2026-05-27_20-52-54
│   ├── 2026-05-27_20-53-45
│   ├── 2026-05-27_20-53-46
│   ├── 2026-05-27_20-53-47
│   ├── 2026-05-27_20-53-49
│   ├── 2026-05-27_21-03-34
│   ├── 2026-05-27_21-23-56
│   ├── 2026-05-27_21-25-20
│   ├── 2026-05-27_21-28-36
│   ├── 2026-05-27_21-31-25
│   ├── 2026-05-27_22-42-22
│   ├── 2026-05-27_22-49-00
│   ├── 2026-05-27_22-49-47
│   ├── 2026-05-27_22-52-18
│   ├── 2026-05-27_22-53-07
│   ├── 2026-05-27_23-22-07
│   ├── 2026-05-27_23-25-55
│   ├── 2026-05-27_23-29-23
│   ├── 2026-05-27_23-41-35
│   ├── 2026-05-28_13-15-19
│   ├── 2026-05-28_13-40-28
│   ├── 2026-05-28_13-43-00
│   ├── 2026-05-28_13-47-12
│   ├── 2026-05-28_13-54-16
│   ├── 2026-05-28_14-02-25
│   └── 2026-06-10_16-00-20
35 directories
user@RaspberrySW:~ $
```

Rysunek 11: Wygląd struktury folderów sesji w oknie terminala.

8. Instrukcja Obsługi i Procedury Terenowe

Krok 1: Uruchomienie i Połączenie

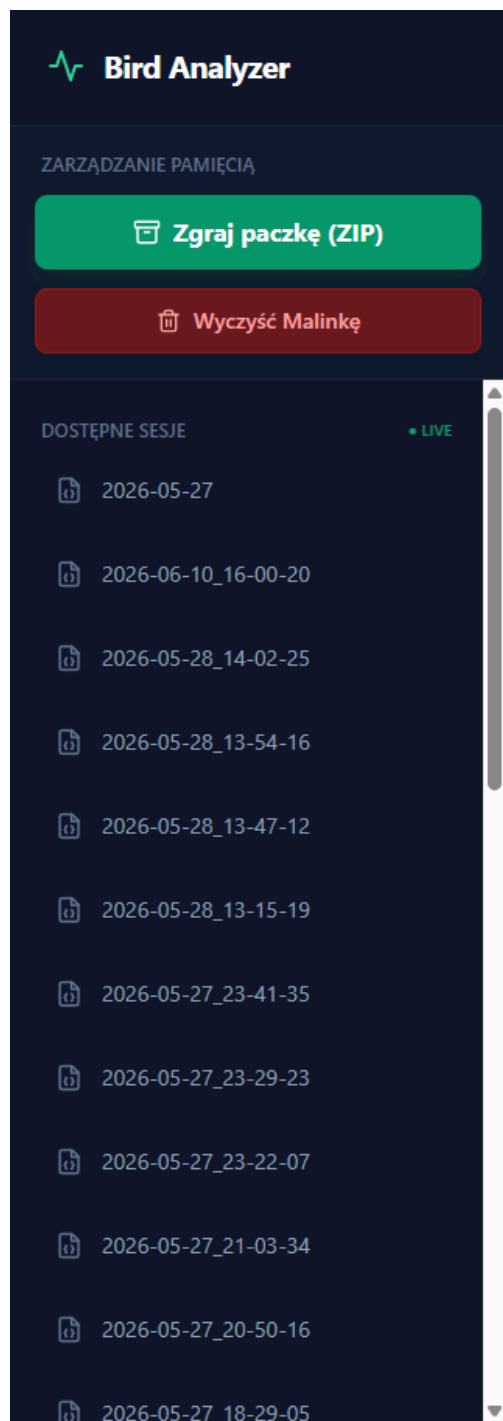
1. Podłącz urządzenie do powerbanka. Oczekaj około 40–60 sekund na uruchomienie systemu.
2. Na telefonie lub komputerze znajdź sieć Wi-Fi o nazwie **DrzewoBirdNET** i zaloguj się hasłem **HasłoDoDrzewa**. Ignoruj komunikat o braku dostępu do internetu.

Krok 2: Przeglądanie danych (Trzy sposoby)

- **Panel WWW (Zalecany):** Otwórz przeglądarkę i wpisz adres `http://10.42.0.1:5173`. Zobaczysz panel użytkownika, w którym z menu bocznego możesz wybrać interesującą Cię sesję, aby przeglądać wykresy i odsłuchiwać nagrania ptaków.
- **Eksplorator Windows:** Wpisz w pasek adresu menedżera plików: `\\10.42.0.1\Raspberry`. Po podaniu loginów użytkownika systemu (user), zyskasz bezpośredni dostęp do folderów na karcie pamięci.
- **Konsola (SSH):** Do celów diagnostycznych możesz połączyć się przez SSH: `ssh user@10.42.0.1`.

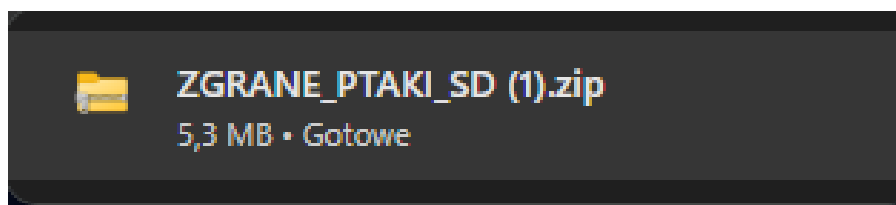
Krok 3: Pobieranie danych i czyszczenie pamięci

Zarządzanie zapisanymi plikami odbywa się w lewym panelu aplikacji:



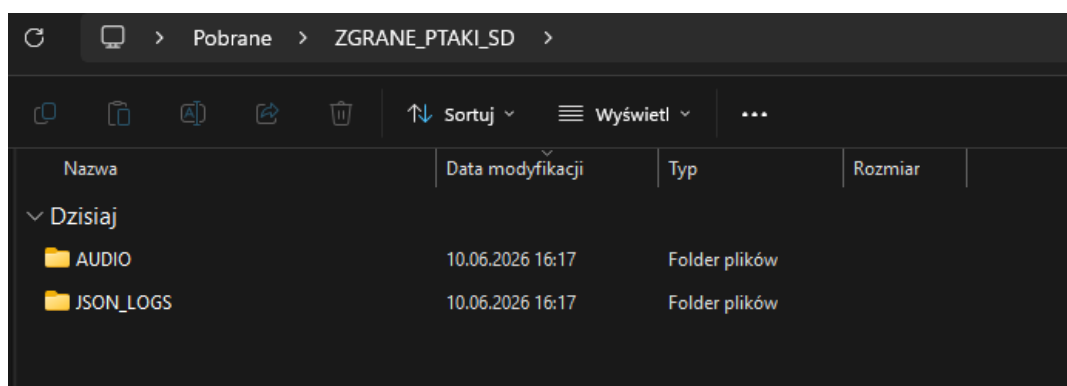
Rysunek 12: Przyciski do pobierania i usuwania danych w menu bocznym.

1. Kliknij zielony przycisk **"Zgraj paczkę (ZIP)"**.
2. System przygotuje i pobierze spakowane archiwum na Twój telefon lub komputer:

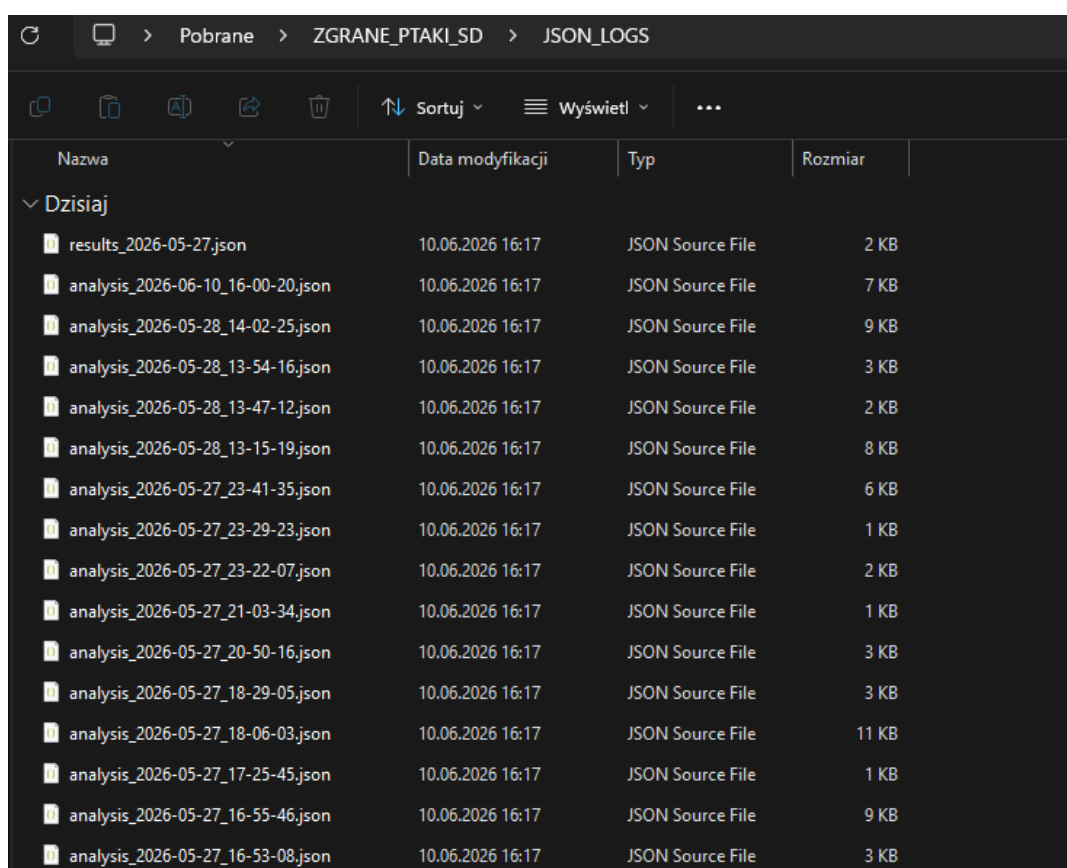


Rysunek 13: Pobieranie wygenerowanej paczki ZIP w przeglądarce.

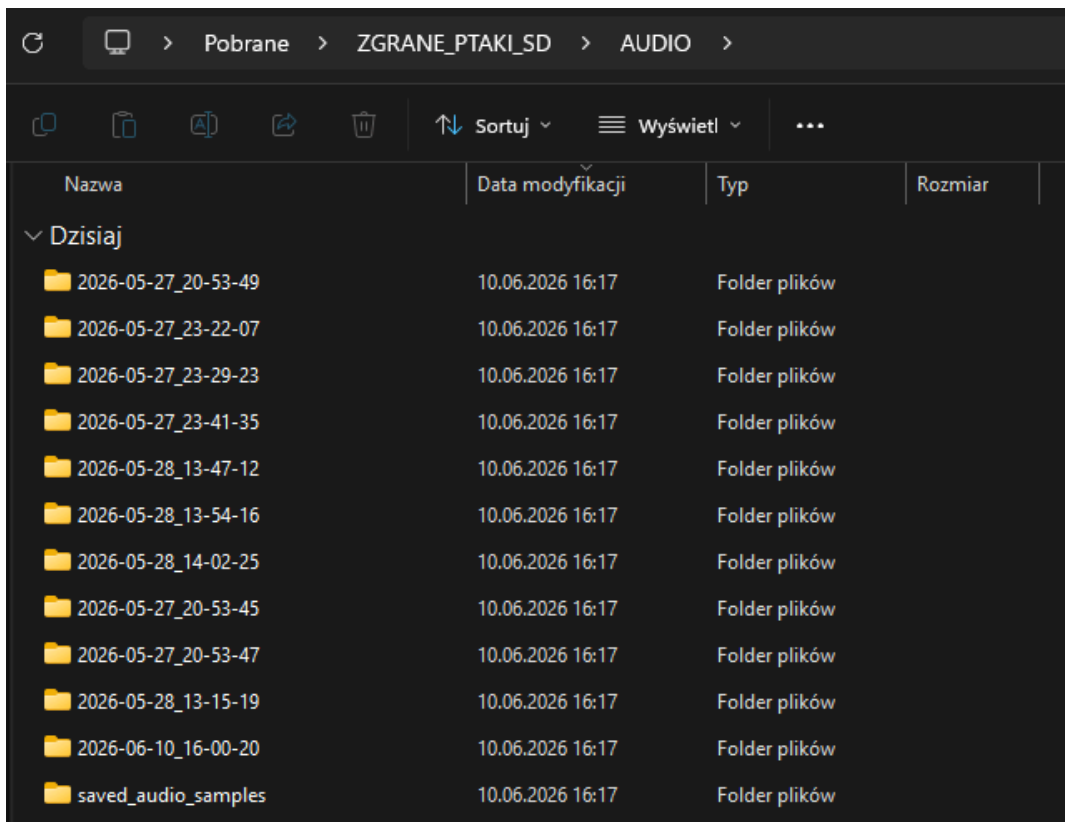
3. Po rozpakowaniu archiwum otrzymasz uporządkowaną strukturę plików z logami i nagraniami:



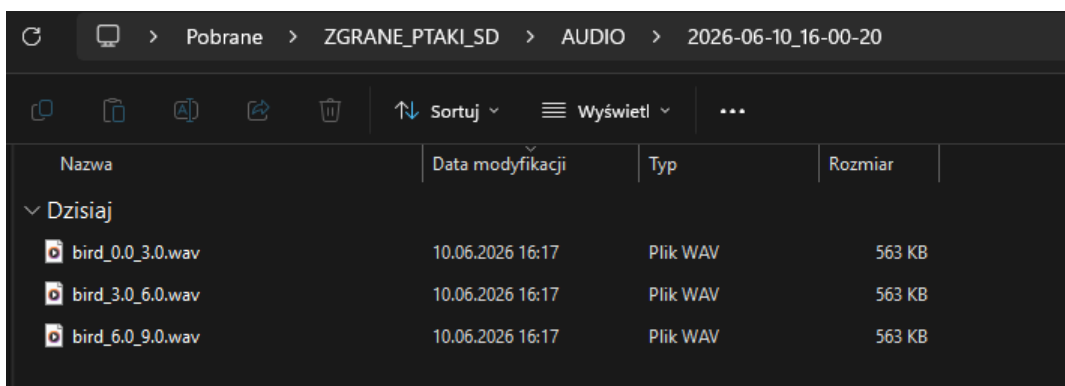
Rysunek 14: Zawartość pobranej paczki ZIP po rozpakowaniu.



Rysunek 15: Zapisane pliki tekstowe JSON z wynikami analizy.

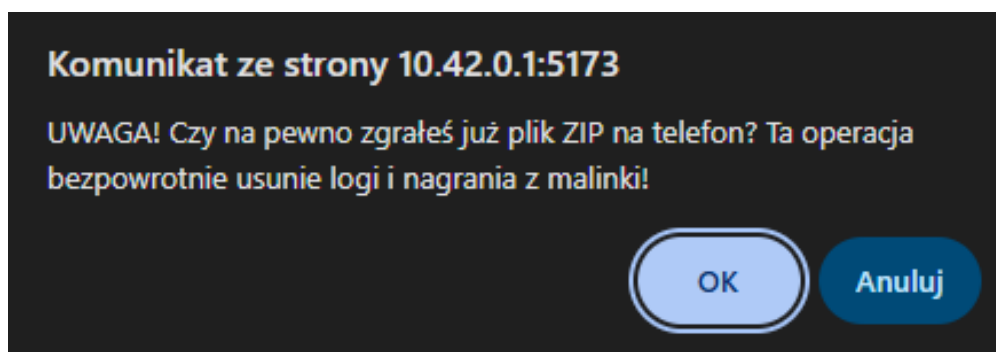


Rysunek 16: Foldery z próbkami dźwiękowymi podzielone na daty.

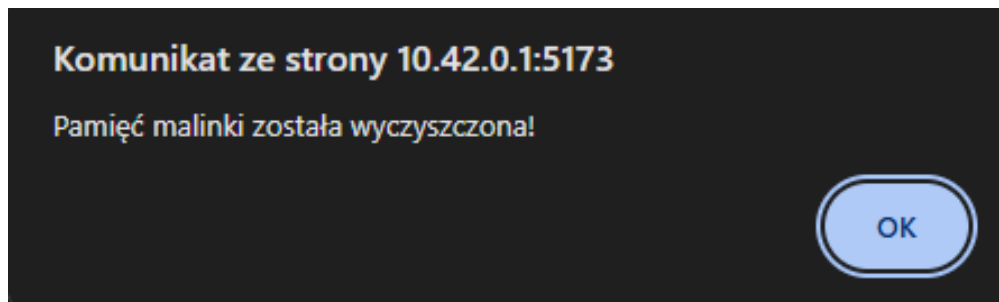


Rysunek 17: Wycięte, krótkie nagrania audio poszczególnych śpiewów ptaków.

- Po upewnieniu się, że pliki zostały w całości zapisane na Twoim komputerze lub telefonie, kliknij czerwony przycisk **"Wyczyść Malinkę"** i potwierdź operację w oknie, które się pojawi:

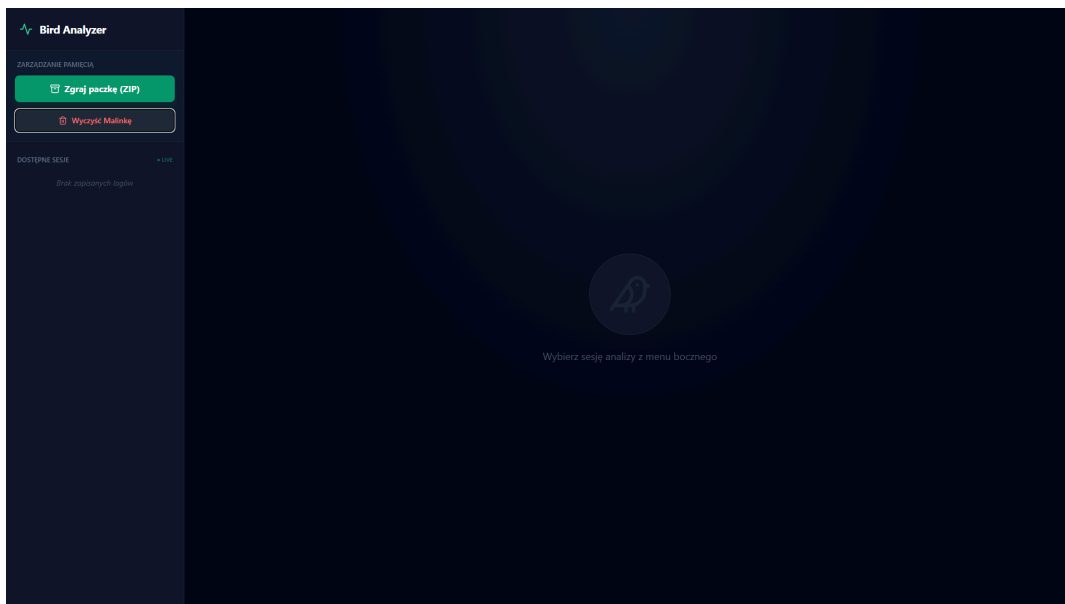


Rysunek 18: Ostrzeżenie przed przypadkowym usunięciem plików.



Rysunek 19: Potwierdzenie pomyślnego usunięcia danych z urządzenia.

5. Karta pamięci zostanie wyczyszczona, a panel wróci do stanu początkowego (brak dostępnych sesji):



Rysunek 20: Widok pustego panelu użytkownika po wyczyszczeniu pamięci.

```
user@RaspberrySW:~ $ tree -d bird_classifier/running/  
bird_classifier/running/  
├── analyzing_results  
├── new_audio_samples  
└── saved_audio_samples  
  
4 directories  
user@RaspberrySW:~ $
```

Rysunek 21: Widok pustej struktury folderów roboczych w terminalu.